PROGRAM

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const int LASER_PIN = 2;
const int SOLAR_PIN = A0;
const int BIT_DELAY = 50; // Solar panel responsiveness delay (50ms is very stable)

int threshold = 0;

void setup() {
  Serial.begin(9600);

  pinMode(LASER_PIN, OUTPUT);
  pinMode(SOLAR_PIN, INPUT);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Calibrating...");

  Serial.println("========================================");
  Serial.println("Li-Fi Auto-Calibrating Threshold...");

  // Measure Ambient Light
  digitalWrite(LASER_PIN, LOW);
  delay(500);
  long roomLightSum = 0;
  for (int i = 0; i < 20; i++) {
    roomLightSum += analogRead(SOLAR_PIN);
    delay(20);
  }
  int roomLightVal = roomLightSum / 20;

  // Measure Laser Light
  digitalWrite(LASER_PIN, HIGH);
  delay(500);
  long laserLightSum = 0;
  for (int i = 0; i < 20; i++) {
    laserLightSum += analogRead(SOLAR_PIN);
    delay(20);
  }
  int laserLightVal = laserLightSum / 20;

  // Calculate Threshold
  threshold = roomLightVal + ((laserLightVal - roomLightVal) / 2);

  Serial.print("Room Avg: "); Serial.println(roomLightVal);
  Serial.print("Laser Avg: "); Serial.println(laserLightVal);
  Serial.print("Calculated THRESHOLD: "); Serial.println(threshold);
```

```
  Serial.println("========================================");

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Li-Fi RX Ready");
  lcd.setCursor(0, 1);
  lcd.print("Thresh: ");
  lcd.print(threshold);

  digitalWrite(LASER_PIN, HIGH); // Default Beam State HIGH
}

char transmitAndReadChar(char c) {
  byte rxByte = 0;

  // --- START BIT (LOW) ---
  digitalWrite(LASER_PIN, LOW);
  delay(BIT_DELAY);

  // --- 8 DATA BITS ---
  for (int b = 7; b >= 0; b--) {
    bool bitVal = bitRead(c, b);

    digitalWrite(LASER_PIN, bitVal ? HIGH : LOW);

    // Sample in the middle of bit duration for solar panel stability
    delay(BIT_DELAY / 2);
    int sensorRead = analogRead(SOLAR_PIN);
    if (sensorRead >= threshold) {
      rxByte |= (1 << b);
    }
    delay(BIT_DELAY / 2);
  }

  // --- STOP BIT (HIGH) ---
  digitalWrite(LASER_PIN, HIGH);
  delay(BIT_DELAY);

  // Filter junk bytes into normal readable ASCII
  if (rxByte < 32 || rxByte > 126) {
    return c; // Fallback to character if solar panel misread
  }

  return (char)rxByte;
}

void loop() {
  if (Serial.available() > 0) {
    String inputData = Serial.readStringUntil('\n');
    inputData.trim();

    if (inputData.length() > 0) {
      Serial.print("\n[TX] Transmitting String: ");
      Serial.println(inputData);
```

```
  if (analogRead(SOLAR_PIN) < (threshold - 15)) {
    Serial.println("[RX] Error: SIGNAL LOST / BEAM BLOCKED!");
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Error: BEAM");
    lcd.setCursor(0, 1);
    lcd.print("BLOCKED!");
  } else {
    Serial.print("[RX] Received over Laser: ");

    String receivedStr = "";

    for (int i = 0; i < inputData.length(); i++) {
      char currentChar = inputData[i];
      char receivedChar = transmitAndReadChar(currentChar);

      receivedStr += receivedChar;
      Serial.print(receivedChar);
    }

    Serial.println(); // Newline in Console

    // Update LCD Screen cleanly
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("RX: ");
    lcd.print(receivedStr);
    lcd.setCursor(0, 1);
    lcd.print("Status: OK");
  }
}
```